

# Take-Home Coding Assignments — Agentica

## General Principles

- **Time limit: 3 hours maximum.** We mean it. A partial solution that shows clear thinking beats a complete solution that took 8 hours. Tell us how long you actually spent.
- **Use any tools you want.** AI coding tools, libraries, Stack Overflow, documentation — whatever you use in real work. We care about the result and your ability to explain it, not whether you typed every character.
- **Submit a working repository.** README with setup instructions, a single command to run. If we can't start it in under 2 minutes, that's a signal.
- **Answer the discussion questions.** These matter as much as the code. Brief, specific answers — a few sentences each.
- **Credentials are for this assignment only.** Do not share API keys, database credentials, or endpoint URLs outside your submission. Do not use them for anything other than this assignment.

## The Task

### Build a streaming chat interface powered by a real LLM.

You're building the frontend for an AI chat product. You'll connect to a real LLM API — the same type of API you'd work with at Agentica. The focus is on the frontend: how you handle real streaming responses, manage conversation state, and build a usable interface.

### You are given access to:

**LLM API** — an OpenAI-compatible API.

- Base URL: `https://llm-test-api.projects.agentica.studio`
- API key: `agentica-test-API`

The API supports the standard `/v1/chat/completions` endpoint with `stream: true`. You'll POST messages and receive a streaming SSE response.

---

### What to build:

1. A chat interface with a message input and a scrolling message history.

2. When the user sends a message, call the LLM API with streaming enabled and render the response token-by-token as it arrives.
3. Support multiple messages in a conversation — maintain conversation history so the LLM sees prior messages for context.
4. Handle edge cases:
  - What happens if the user sends an empty message?
  - What happens if the API errors or the stream drops mid-response?
  - What happens if the user sends a new message while a response is still streaming?
  - Loading states and visual feedback.

### Architecture decision — your call:

You can call the LLM API directly from the frontend, or you can write a minimal backend proxy. Both approaches are valid. If you call the API directly from the browser, the API key will be visible in client-side code — this is acceptable for this assignment. In your discussion answer, explain what you'd do differently in production.

### Constraints:

- React + TypeScript. Vite preferred.
- Tailwind CSS or any styling approach. The UI should be **usable** — user messages and AI messages must be visually distinct.

### Bonus (only if you have time):

- A "stop generating" button that cancels the stream mid-response.
- Auto-scroll that follows the streaming text but stops if the user scrolls up.
- Markdown rendering in AI responses.
- Conversation history in the sidebar (multiple chat sessions).

### Discussion Questions (answer in your README)

1. **AI Dev Stack:** What AI tools were used in this assignment? Which models do you use?
2. **API discovery:** How did you find the model name? Walk through your process.
3. **Architecture:** Did you call the API directly from the browser or through a proxy? Why? What would you change in production?
4. **Streaming implementation:** How did you parse the SSE stream? What library or approach did you use, and what were the alternatives?
5. **State management:** How did you manage conversation history and the streaming response state? Why that approach?
6. **Tradeoffs:** What did you skip or simplify? What would you add first with another 3 hours?

7. **Time spent:** How long did you actually spend? What took longer than expected?